

Курс лекций

Объектно-ориентированное программирование

2026 г

ФГБОУ ВПО КубГУ

Зав. каф. ВМиИ, к.ф.-м.н,
доцент, Фоменко С.И.

Тема 1:
Введение в язык программирования C++

1. История языка программирования C++
2. Введение в программирование на C++
3. Консольный ввод-вывод
4. Переменные и типы данных
5. Операторы
6. Массивы и указатели

C++ (си-плюс-плюс) — компилируемый, *статически типизированный язык* программирования

Статически типизированный язык означает, что переменные ранее объявленного типа не могут менять свой тип во время исполнения блока команд.

История

Язык С («Си») разработан в 1969-1973 годах Кеннетом Томпсоном и Деннисом Ритчи. Официальной датой появления языка С считается 1972 год. Операционной системы UNIX написана на С.



Кеннет Томпсон



```
#include <stdio.h>
main()
{
      printf("hello, world\n");
}
```

Dennis M. Ritchie (1941-2011)

История

Язык C++ -- усовершенствованная версия языка C, в котором включена поддержка ООП (C + Simula). Язык C++ был разработан Бьёрном Страуструпом в 1983 году.

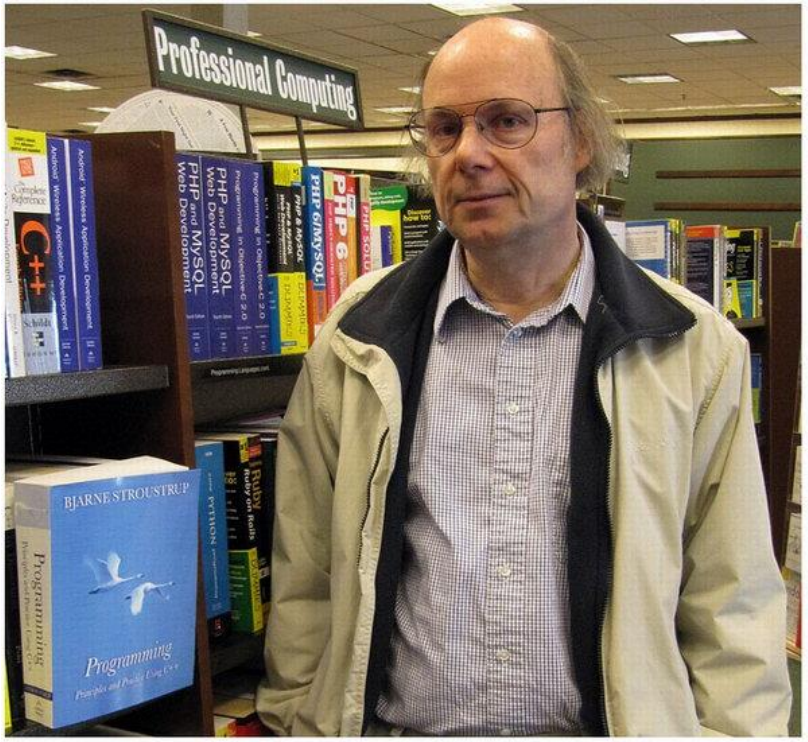


На языке C++ написана ОС Windows.

История

Бьерн Страуструп:

C++ имел и имеет по сей день те преимущества, которые предоставляет C в низкоуровневом программировании при работе с аппаратной частью (проще говоря, «железом»), и в то же время обеспечивает возможность создавать собственные уровни абстракции без потери излишней избыточности в плане использования ресурсов. Ведь конструкторы и деструкторы стали в свое время настоящим прорывом. По сравнению с традиционным C (Кернигана и Ритчи), C++ предлагает улучшенную производительности и систему безопасности типов. По сравнению с Simula, C++ существенную гибкость и эффективность.



Бьерн Страуструп. Программирование: принципы и практика использования C++



Преимущества языка

• C++ — современный язык. Он включает в себя управляющие конструкции, рекомендуемые теоретическим и практическим программированием.

• C++ — эффективный. Структура позволяет наилучшим образом использовать возможности современных ЭВМ. Программы отличаются компактностью и быстротой исполнения.

• C++ — переносимый или мобильный. Программа, написанная для одной вычислительной системы, может быть перенесена почти без изменений на другую. Компиляторы реализованы почти на 40 типах вычислительных систем, начиная от 8-ми разрядных процессоров и кончая CRAY-1 один из мощных суперкомпьютеров.

• C++ — мощный и гибкий. Большая часть ОС UNIX написана на C.

• C++ — удобный язык. Достаточно структурирован, чтобы поддерживать хороший стиль программирования, в то же время не накладывает больших ограничений.

• C++ — язык " компилирующего" типа. А значит получаются более быстрые по исполнению программы.

• Java, JavaScript, C#, как и огромное количество других популярных языков программирования, содержит в основе принципы C++.



Отличия C++ от C

Совместимость: По большей части C++ является надстройкой над Си, поэтому большая часть кода на Си будет компилироваться и в C++, но в обратную сторону это не работает

Применение: Си используется в системном программировании, микроконтроллерах и операционных системах (Linux). C++ применяется там, где важна скорость и сложные структуры данных: в игровых движках, браузерах и тяжелом софте с пользовательским интерфейсом.



Отличия C++ от C

Характеристика	Язык Си (C)	Язык C++
Парадигма	Процедурное программирование	Мультипарадигменный (ООП, обобщенное и процедурное)
Подход к ООП	Нет классов и объектов (эмулируется вручную)	Есть встроенные классы, объекты, наследование и полиморфизм
Библиотека	Базовая стандартная библиотека (stdio.h, stdlib.h)	Огромная библиотека STL (std::vector, std::string, алгоритмы)
Управление памятью	Ручное через функции malloc и free	Ручное + безопасное через **new/delete**, умные указатели (**std::unique_ptr**) и конструкторы
Перегрузка функций	Не поддерживается	Поддерживается (одинаковые имена функций с разными аргументами)

Сравнение ввода-вывода в Си и C++

Особенность	Язык Си (C)	Язык C++
Библиотека	<stdio.h>	<iostream>
Инструменты	Функции printf(), scanf()	Объекты потоков std::cout, std::cin
Синтаксис	Операторы вызова функций ()	Операторы сдвига << (вставка) и >> (извлечение)
Типобезопасность	Нет. Нужно вручную указывать спецификаторы типа (%d, %f, %s). Ошибка в спецификаторе приводит к сбою.	Да. Компилятор автоматически определяет тип переменной на этапе компиляции.
Расширяемость	Нельзя напрямую передать в printf пользовательскую структуру данных.	Можно перегрузить операторы << и >> для вывода собственных классов.
Скорость (по умолчанию)	Быстрее. Работает напрямую с буфером ввода-вывода.	Медленнее, так как потоки iostream по умолчанию синхронизируются с stdio.

Сравнение ввода-вывода в Си и C++. Примеры кода

Ввод и вывод на языке Си:

```
#include <stdio.h>int
main() {
int age;
printf("Введите возраст: ");
scanf("%d", &age);
printf("Вам %d лет.\n", age);
return 0;}
```

Ввод и вывод на языке C++

```
#include <iostream>
int main() {
int age;
std::cout << "Введите возраст: ";
std::cin >> age;
std::cout << "Вам " << age << " лет." << std::endl;
 return 0;
}
```



Отличия C++ от C

Ввод и вывод на языке C++

```
#include <iostream>
int main() {
int age;
std::cout << "Введите возраст: ";
std::cin >> age;
std::cout << "Вам " << age << " лет." << std::endl;
 return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
setlocale(LC_CTYPE, "Russian");

int age;
cout << "Введите возраст: ";
cin >> age;
cout << "Вам " << age << " лет." << std::endl;
 return 0;
}
```

Операторные скобки

```
{
   что-то
}
```

Примеры:

```
int main()
{
   тело программы
}
```

```
if(2>1)
{
 значит что-то делаем
}
```

Разделение операторов: точка с запятой

```
оператор1 ; оператор2 ;
```

Пример:

```
a=2;
b=a+1;
```

Программа (главный исполняемый модуль)

Примеры:

```
int main()
{
   int a,b,c;

   a=1;
   b=2;
   c=a+b;
   return 0;
}
```

```
int – встроенный тип данных
      для целых чисел
a,b,c - переменные

return val – выход из
 программы (процедуры) и
 возврат значения val
```

Комментарии

```
// однострочный комментарий
```

```
/* многострочный
   комментарий
*/
```

Комментарии

Примеры:

```
/* программа вычисления суммы
   двух чисел
*/
int main()
{
  int a,b,c; // объявления переменных

  a=1;     // присвоение значения переменной a
  b=2;     // присвоение значения переменной b
  c=a+b; // вычисление суммы
  return 0; // выход из программы без ошибок
}
```



Основы: заголовочные файлы

Заголовочный файл **имя_библиотеки.h** имеет расширение .h

Включение заголовочного файла (условно библиотеки)

```
#include <имя_библиотеки>
```

Примеры:

```
#include <iostream>

int main()
{
   тело программы
}
```



Пространство имен

Пространства имен – своего рода модуль, который объединяет переменные, процедуры. Позволяет исключить пересечение имен.

Доступ к члена пространства имен

```
Имя_пространства::имя_компоненты
```

Примеры:

```
#include <iostream>

int main()
{
 std::cout << "Ура! Заработало!";/* вывод на
 экран */
}
```

Прямой доступ к члена пространства имен

```
using  Имя_пространства::имя_компоненты
```

Примеры:

```
#include <iostream>

int main()
{
  using std::cout; // открыт доступ к cout
  cout << "Ура! Заработало!";  // вывод на экран
}
```

Прямой доступ ко всем членам пространства имен

```
using namespace  Имя_пространства
```

Примеры:

```
#include <iostream>
int main()
{
  int a;
  using namespace std; // полный доступ к std
  cout << "Введите a: ";  // вывод на экран
  cin >> a;  // ввод с экрана
  // тоже самое что: std::cin >> a;
  return 0;
}
```

Ввод и вывод на экран

1. Подключить заголовочный файл `iostream.h`

```
#include <iostream>
```

2. Вывод на экран:

```
std::cout << "строка" << переменная << std::endl
```

```
std::cout – стандартный объект вывода
<<  – оператор отправки переменной для вывода
std::endl – конец строки, перевод каретки ("\n")
```

3. Ввод на экран:

```
std::cin >> переменная
```

```
std::cin – стандартный объект ввода
>>  – оператор отправки переменной для ввода
```



Ввод и вывод на экран

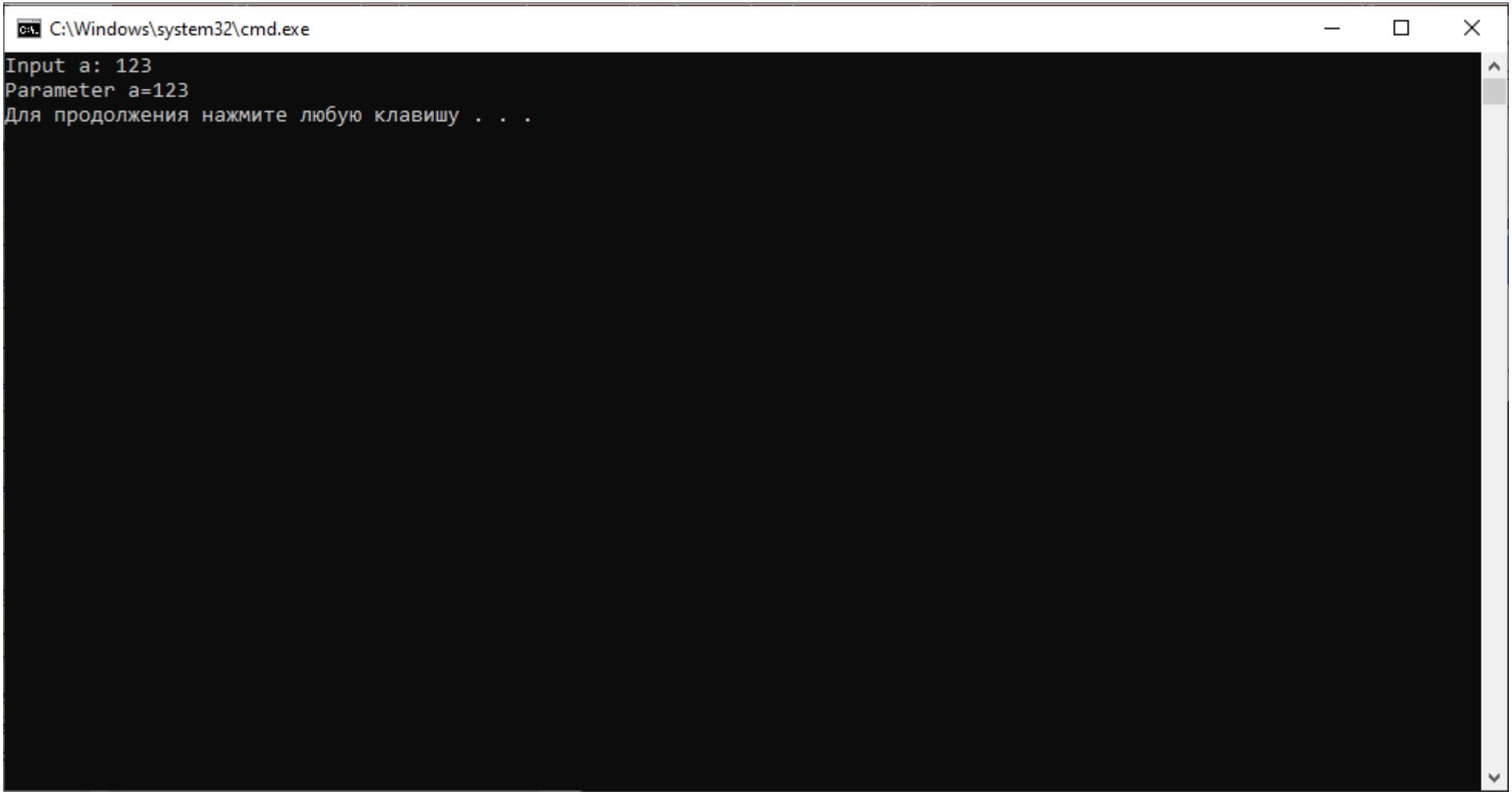
Пример:

```
#include <iostream>

int main()
{
   int a;
   using namespace std; // полный доступ к std
   cout << "Input a: ";  // вывод на экран
   cin >> a;  // ввод с экрана
   cout << "Parameter a=" << a << endl;
   return 0;
}
```

Ввод и вывод на экран

Пример:



Объявление переменных определенного типа

```
[спецификатор] тип_данных   имя_переменной
```

Присваивание

```
имя_переменной = выражение_или_значение
```

Примеры:

```
int main()
{
  int a;
  a = 5;
  const double b = 2.5; // const => b--постоянная
  double c = a*b;
  return 0;
}
```

Имя переменной

1. В именах разрешено использовать только алфавитных символов, цифр и символа подчеркивания (_).
2. Символы в верхнем и нижнем регистре рассматриваются как разные.
3. Первым символом имени не должна быть цифра.
4. Первым символом имени не рекомендуется использовать символ подчеркивания (_).
5. В качестве имени нельзя использовать ключевое слово C++.

```
int po3; // допустимое
int Po3; // допустимое и отличающееся от po3
int PO3; // допустимое и отличающееся от двух предыдущих
int 4ever;// недопустимое, потому что начинается с цифры
int double;// недопустимое:double является ключевым словом C++
int begin; // допустимое:begin является ключевым словом Pascal
```

Встроенные типы данных

	Тип	Диапазон значений	Размер (байт)
Логический тип →	bool	true и false	1
Символьные типы	signed char	-128 … 127	1
	unsigned char или **char**	0 … 255	1
Целочисленные типы	signed short int или **int** для 16x	-32 768 … 32 767	2
	unsigned short int	0 … 65 535	2
	signed long int или **int** для 32x	-2 147 483 648 … 2 147 483 647	4
	unsigned long int	0 … 4 294 967 295	4
	long long (или long long int)	−9 223 372 036 854 775 808 … +9 223 372 036 854 775	8
Типы с плавающей точкой	float	3.4e-38 … 3.4e+38	4
	double	1.7e-308 … 1.7C+308	8
	long double	3.4e-4932 … 3.4e+4932	10
Пустой тип →	void	без значения	

Автоматическое определенного типа (стандарт C++11)

```
auto  имя_переменной = значение_выражение
```

Примеры:

```
int main()
{
 int a;
 using namespace std; // полный доступ к std
 cout << "Input a: ";  // вывод на экран
 cin >> a;  // ввод с экрана
 double b = 2.5;
 auto c = a*b;  // ⇔ double c=a*b;
 cout << "c=a*b: " << c << endl;  // вывод на экран
 return 0;
}
```



Операторы

Арифметические операторы

Оператор	Описание	Пример
+	Сумма двух операндов	2 + 1 даст 3
-	Вычитание	2 – 1 даст 1
*	Умножение	2 * 3 даст 6
/	Деление (зависит от типа)	4 / 2 даст 2
%	Деление по модулю	3%2 даст 1
++	Инкремент, увеличивает значение на единицу	3++ даст 4
– –	Декремент, уменьшает значение на единицу	3– – даст 2



Операторы

Операторы сравнения: результат bool (true, false)

Оператор	Описание	Пример
==	Равенство между операндами	(1 == 2) даст false.
!=	Неравенство межу операндами	(1! = 2) даст true.
>	Операнд 1 больше операнда 2.	(1> 2) даст false.
<	Операнд 1 меньше операнда 2.	(1 <2) даст true.
> =	Операнд 1 больше или равен операнду 2.	(1> = 2) даст false.
<=	Операнд 1 меньше или равен операнду 2.	(1 <= 2) даст true.



Операторы

Логические операторы: результат bool (true, false)

Оператор	Описание	Пример
&&	Логическое И	(true && false) даст false.
||	Логическое ИЛИ	(true || false) даст true.
!	Логическое отрицание	! (true && false) даст true.

Побитовые операторы

Оператор	Описание	Пример
&	Побитовое И	(2#101 & 2#011) даст 2#001 = 1
|	Побитовое Или	(2#101 | 2#011) даст 2#111 = 7
^	Исключающее Или (XOR)	(2#101 ^ 2#011) даст 2#110 = 6
~	Бинарное отрицание	(~ 2#101) даст 2#010 = 2
<<	Битовый сдвиг влево первого операнда на количество бит, заданного вторым операндом	2#0101 << 1 даст 2#1010 = 10
>>	Битовый сдвиг вправо первого операнда на количество бит, заданного вторым операндом	2#0101 >> 1 даст 2#-0010 = 2

Операторы присваивания

Оператор	Описание	Пример
=	Простой оператор присваивания, присваивает значения из правого операнда левому операнду	C = A + B присваивает значение A + B в C
Бинарный_операнд =	Выполняет бинарный операнд с левым и правым операндом и присваивает значение левому операнду	C + = A эквивалентно C = C + A C –=A ⇔ C = C – A C*=A ⇔ C = C*A

Другие операторы

Оператор	Описание	Пример
условие ? X : Y	Если Условие истинно, то возвращается X, иначе Y.	(1>2) ? 4 : 5 даст 5
тип()	Преобразование (явное приведение) типа	int a; a=int(2.2); // a=2
&	Возвращает адрес переменной.	&x даст фактический адрес переменной.

*	1) Объявление указателя
	2) Возвращает значение переменной по адресу, закрепленному за ссылкой (разыменование).
		int *p;
		int x= 2;
		p=&x;
		int y=*p
		даст y=2

Другие операторы

Оператор	Описание	Пример
new	Выделение памяти и присвоение адреса указателю p = new тип или p = new тип(инициализатор) p = new тип[size] -- одномерный массив	double *a a = new double[10] int b=new int int c=new int(1) // c =1, b=0, a -- массив
delete	Освобождение ранее выделенной памяти оператором new	delete[]a; delete b; delete c;
sizeof	Возвращает размер переменной.	sizeof(2.0) даст 4 sizeof('a') даст 1

Условный оператор if .. else

```
if (bool_условие) один_оператор;
```

```
if (bool_условие)
{
 // блок операторов , если верно
}
```

```
if (bool_условие)
{
  // блок операторов, если верно
}
else
{
 // блок операторов, если неверно
}
```

Условный оператор if .. else

Примеры:

```
float x=1,y=2;

if (x<y)
{
    cout << "x<y"
}
else
{
    cout << "x>=y"
}
```

Оператор выбора switch

```
switch (выражение)
{
    case val_1:
         тело1;  // операторные скобки не нужны
         break; // выход из switch
    case val_2:
         тело2;
         break; // выход из switch
    …
    case val_3:
           тело3;
    case val_4:
          тело4; // для случая val_3 также выполняется!
         break; // выход из switch
    default:
         действия по умолчанию; // необязательно
}
```

Оператор цикла с параметрами (for)

```
for(инициализация; условие; итерируемое выражение
после основного тела цикла)
{
  тело цикла
}
```

Примеры:

```
float S=0;
int n=10;

for(int i=1; i <= n; ++i)
{
  S+=1.0/(i*i); // 1/(i*i) – неверно. Почему?
}
```

Оператор цикла с предусловием (while)

```
while(условие)
{
   тело цикла, если условие выполняется
}
```

Примеры:

```
float S=0;
int n=10, i=0;

while(i<n)
{
   i++;
   S+=1.0/(i*i);
}
```

Оператор цикла с постусловием (do …while)

```
do
{
   тело цикла, если условие продолжает выполняется
}
while(условие)
```

Примеры:

```
float S=0;
int n=10, i=0;

do
{
   i++;
   S+=1.0/(i*i);
}
while(i<n);
```

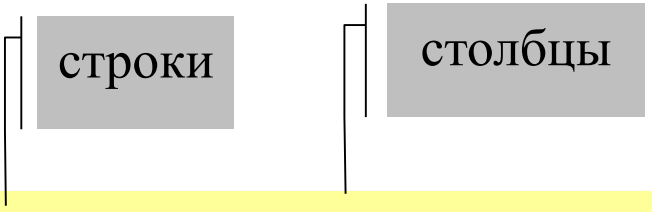
Массивы с известным размером

```
тип   имя_переменной[размер]
```

Обращение к элементам массива

```
имя_переменной[index],      0<=index<размер
```

Многомерные массивы




```
тип   имя_переменной[размер1][размер2]
```

Обращение к элементам

```
имя_переменной[index1][index2]
0<= index1 < размер1, 0<= index2 < размер2,
```

Многомерные массивы

строки

столбцы

```
тип   имя_переменной[размер1][размер2]
```

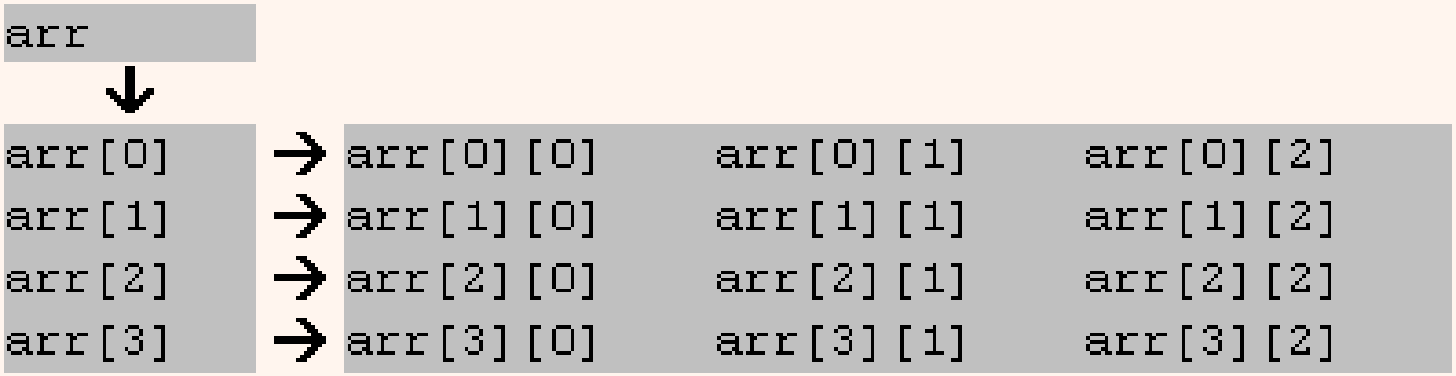


Рис.3. Распределение памяти для двумерного массива.

Пример:

```
int main()
{
  int   a[2]; // вектор длины 2
 const int sz1 = 3, sz2 = 2;
 float b[sz1][sz2]; // массив 3x2

 a[0]=2; a[1]=2;
 b[0][0]=1.0+a[0]; // b[0][2]=3.0
 b[0][1]=2.0+a[1]; // b[0][1]=4.0

 using namespace std;
 cout << "b[0][0]= " << b[0][0] << endl;

 return 0;
}
```

Конструкторы массивов

```
тип  имя_переменной[n]={val_1, val_2,…, val_n}
```

Автоматическое определение размера при инициализации

```
тип  имя_переменной[]={val_1, val_2,…, val_n}
```

Конструкторы многомерных массивов

```
type  имя_переменной[n][m] = {{val_11,
val_12,…, val_1m},…, {val_n1, val_n2,…,
val_nm}}
```

Пример:

```
int a[]={1,2};
int b[2][2]={{11,12},{21,22}};
int c[][2]={{11,12},{21,22}};
```

Указатель

```
тип   *имя_указателя
```

Присвоение адреса указателю

```
имя_указателя = адрес
```

```
имя_указателя = &переменная
```

Арифметика указателей

Смещение указателя на ± n*sizeof(тип) байт (n – целое)

```
имя_указателя = имя_указателя + n   ⇔ имя_указателя +=n
имя_указателя = имя_указателя - n;  ⇔ имя_указателя -=n
++имя_указателя;  --имя_указателя;
```

Обращение к переменой, на которую ссылается указатель

```
*имя_указателя
```

Примеры:

```
int   a[2]; // вектор длины 2
int   *p;

p = &a[0]; // ссылка на 1 элемент a
*p = 1;  // ⇔ a[0]=1
p++; // перемещение на 1 позицию вправо
*p = 2; // ⇔ a[1]=2
```

или:

```
p = a; // ⇔ a - хранит адрес &a[0];
*p = 1;  // ⇔ a[0]=1
p++; // перемещение на 1 позицию вправо
*p = 2; // ⇔ a[1]=2
```

или: работа с указателями как с массивом

```
int   a[2]; // вектор длины 2
int   *p;

p = a; // ⇔ a – хранит адрес &a[0];
p[0] = 1;  // ⇔ a[0]=1
p[1] = 2; // ⇔ a[1]=2
```

Пример с двумерным массивом: НЕ РАБОТАЕТ!

```
int   a[2][2]={{1,2},{3,4}};
int   **p; // указатель «на указатели»
(указатель на массив массивов)
p = a;
cout << p[1][1]; // ⇔ cout << a[1][1],
```

Выделение памяти для одномерного массива

```
тип  *имя_указателя = new тип[размер]
```

Удаление одномерного динамического массива

```
delete[] имя_указателя
```

Пример:

```
int m = 2;
int  *a = new int[m]; // динамический массив длины 2
a[0] = 1;
a[1] = 2;
…
delete[] a;          // освобождение памяти
…
a = new int[2*m]; // динамический массив длины 4
```

Литература

1. Липпман, С.Б. Язык программирования C++. Базовый курс, -- М.: Вильямс, 2014 – 1120 с.
2. Сиддхартха Рао. Освой самостоятельно C++. – М: Вильямс., 2013. – с 651.
3. Бьерн Страуструп. Программирование: принципы и практика использования C++ – М: Вильямс, 2011. – 1248 с.: